

# Vave Geospatial Risk Pipeline (H3 / Databricks)

This repository implements a Databricks-style Spark pipeline that ingests risk events, indexes them with Uber H3, and publishes daily risk aggregates by H3 cell and “zone” (cell + neighbours) using a Medallion architecture (Bronze → Silver → Gold).

## Assumptions

- Processing mode: batch / micro-batch on top of structured streaming (no hard real-time SLA).
- Time granularity: daily (event\_dt), derived from event\_timestamp; queries can still filter by full timestamp.
- Typical volume: designed to handle at least tens of thousands of events per day; patterns scale to 100M+ with tuning (see “Scaling” below).
- Geography: events clustered in a city-scale region (e.g. London), where a mid-range H3 resolution is appropriate.

## How to run

### Prerequisites

- Databricks workspace with Unity Catalog and built-in H3 functions (DBR with H3 support).
- Catalog main with schemas: control, \_raw, bronze, silver, gold.
- Volume for raw CSV files, e.g. /Volumes/main/\_raw/external\_data/vave\_raw.

### Steps

#### 1. Upload sample data

Upload the sample files into the raw volume, for example:

- Vave\_eventsfeed\_20260508.csv
- Vave\_eventsfeed\_20260509.csv
- Vave\_eventsfeed\_20260510.csv

#### 2. Load configuration

Run Vave\_Config\_Params.ipynb:

- Defines static configuration (catalog, schemas, H3 resolution, folder names).
- Exposes dynamic parameters via widgets (file type, file name prefix, file paths, batch id).
- Prints effective paths and table names, including:
  - Bronze: main.bronze.vave\_eventsfeed

- Silver: `main.silver.vave_eventsfeed`
- Gold: `main.gold.daily_risk_cell`, `main.gold.daily_risk_zone`.

### 3. Ingest into Bronze

Run `Vave_Bronze_dbt.ipynb`:

- Reads new CSV files from the configured raw volume path.
- Writes them into `main.bronze.vave_eventsfeed` as an append-only Delta table.
- Adds file-level metadata (e.g. source filename) and `ingestion_ts` for replay/debug.

### 4. Build and maintain Silver

Deploy `silver_vave_eventsfeed.py` as a Databricks pipeline step or streaming job:

- Reads `main.bronze.vave_eventsfeed` as a stream.
- Casts types and derives `event_dt = DATE(event_timestamp)`.
- Derives H3 index: `h3_ix = h3_longlatash3string(longitude, latitude, h3_res)`.
- Applies data-quality expectations and drops invalid rows.
- Writes an incrementally updated Delta table `main.silver.vave_eventsfeed` partitioned by `event_dt`, using CDC on `event_id` and `ingestion_ts` (SCD Type 1).

### 5. Build Gold aggregates

Deploy and run:

- `gold_daily_risk_cell.py` → creates/refreshes `main.gold.daily_risk_cell`.
- `gold_daily_risk_zone.py` → creates/refreshes `main.gold.daily_risk_zone`.

### 6. Data quality checks (optional but recommended)

- Run the data-quality notebook or equivalent checks to validate Bronze → Silver → Gold reconciliation and data freshness.

For local Spark, you can point the same scripts at local CSV/Delta paths and create equivalent tables.

## Repository structure (Task 1)

- `README.md`  
High-level summary of the solution.
- `Candidate-Case-Study-H3-Data-Engineer.docx`  
Original specification for the exercise.
- `Vave_Config_Params.ipynb`  
Central configuration/parameter notebook (widgets + derived paths/tables).
- `Vave_Setup.ipynb`  
Notebook shell intended as a setup/orchestration entry point.

- `Vave_Bronze_dbt.ipynb`  
Bronze-layer ingestion from CSV files in ADLS Gen2 (Volumes) into `main.bronze.vave_eventsfeed`.
  - `silver_vave_eventsfeed.py`  
Silver-layer transformation using `pyspark.pipelines`:
    - Streaming read from Bronze.
    - Data-quality expectations.
    - H3 enrichment.
    - SCD Type 1 (“latest-wins”) semantics per `event_id`.
  - `gold_daily_risk_cell.py`  
Gold-layer daily risk aggregation by H3 cell, writing `main.gold.daily_risk_cell`.
  - `gold_daily_risk_zone.py`  
Gold-layer daily risk aggregation by H3 “zone” (centre cell + neighbours), writing `main.gold.daily_risk_zone`.
  - `Vave_eventsfeed_20260508.csv`, `Vave_eventsfeed_20260509.csv`,  
`Vave_eventsfeed_20260510.csv`  
Sample input data matching the expected event schema.
- 

## H3 indexing: logic and resolution choice (Task 2)

### Where H3 is applied

- H3 indexing is applied in `silver_vave_eventsfeed.py` when building `main.silver.vave_eventsfeed`.
- The pipeline computes:
 

```
F.expr("h3_longlatash3string(longitude, latitude,
                              h3_res)").alias("h3_ix")
```
- Silver-level fields include:
  - `event_id`, `event_timestamp`, `event_dt`
  - `latitude`, `longitude`, `risk_score`
  - `source_file_name`, `ingestion_ts`
  - `h3_ix` (H3 spatial index string)

### H3 resolution and rationale

- `h3_res` is configured in `Vave_Config_Params.ipynb` (e.g. `h3_res = 8`).
- A mid-range resolution is chosen because:
  - It yields neighbourhood-sized hexagons for city-scale data.
  - It balances:
    - **Accuracy** – captures local risk variation.
    - **Performance** – keeps the number of cells manageable.

- **Business interpretation** – a “risk zone” is meaningful as a local area, not just a single point or an entire region.

## Effect of changing resolution

- **Lower (coarser) resolution:**
    - Larger cells, fewer total cells.
    - Faster aggregations and smaller tables.
    - Blunter “risk zones” that may hide local hotspots.
  - **Higher (finer) resolution:**
    - Smaller cells, many more cells.
    - More precise risk localisation.
    - Sparser data and heavier joins/storage.
- 

## Risk aggregation (Task 3)

### Daily risk per H3 cell

`gold_daily_risk_cell.py` builds `main.gold.daily_risk_cell` from `main.silver.vave_eventsfeed`:

- Grouped by `event_dt` and `h3_ix`.
- Metrics:
  - `event_count` (COUNT)
  - `total_risk_score` (SUM)
  - `avg_risk_score` (AVG)

This satisfies the requirement to aggregate by H3 cell and by time window (daily).

### Behaviour at different H3 resolutions

- **Coarser H3:**
  - Fewer, larger cells.
  - Smoother risk metrics, suitable for high-level reporting.
- **Finer H3:**
  - Many small cells.
  - More detailed hotspots but potentially noisy and sparse.

### Handling low-density / empty cells

- Empty cells do not appear in `main.gold.daily_risk_cell` at all.
  - Low-density cells will have low `event_count`; downstream consumers can:
    - Filter on `event_count` thresholds.
    - Use `main.gold.daily_risk_zone` for smoothed metrics across neighbouring cells.
-

## Zone-level risk (Task 4)

### Definition and implementation

Zone-level risk is defined as:

- A centre H3 cell plus its immediate neighbours (k=1 ring).

`gold_daily_risk_zone.py` implements this:

- Reads `main.gold.daily_risk_cell` as `risk_cell`.

- Uses:

```
F.explode(F.expr("h3_kring(h3_ix, 1)").alias("member_h3"))
```

to expand each cell into its k=1 ring (centre + neighbours).

- Joins back to `risk_cell` on `(event_dt, member_h3)` and aggregates per `(event_dt, zone_id)`:
  - `contributing_cells`
  - `event_count`
  - `total_risk_score`
  - `avg_cell_risk_score`

### Double-counting and production handling

- Because each cell can appear in multiple zones, summing `total_risk_score` over all zones double-counts risk.
  - In this solution:
    - `main.gold.daily_risk_cell` is the authoritative source for global risk totals.
    - `main.gold.daily_risk_zone` is used for ranking, hotspot identification, and local context.
  - In a production system, if non-overlapping aggregates are required at a higher level, you can:
    - Define “super-zones” at a coarser H3 resolution.
    - Map fine-grain cells to a single parent zone before aggregating.
- 

## Output model and example queries (Task 5)

`main.gold.daily_risk_cell` – daily risk by cell

Core schema:

- `event_dt` DATE (partition key)
- `h3_ix` STRING
- `event_count` BIGINT
- `total_risk_score` DOUBLE
- `avg_risk_score` DOUBLE

### Example: risk trend over time for a given location

```
WITH target AS (  
  SELECT h3_longlatash3string(-0.12, 51.5, 8) AS target_h3  
)  
SELECT  
  event_dt,  
  total_risk_score,  
  event_count  
FROM main.gold.daily_risk_cell  
CROSS JOIN target  
WHERE h3_ix = target.target_h3  
ORDER BY event_dt;
```

This shows the daily risk trend for the H3 cell that covers a given latitude/longitude.

### main.gold.daily\_risk\_zone – daily risk by zone

#### Core schema:

- event\_dt DATE (partition key)
- zone\_id STRING (centre H3 cell)
- contributing\_cells BIGINT
- event\_count BIGINT
- total\_risk\_score DOUBLE
- avg\_cell\_risk\_score DOUBLE

### Example: top 10 riskiest zones in the last 24 hours

```
SELECT  
  zone_id,  
  event_dt,  
  total_risk_score,  
  event_count  
FROM main.gold.daily_risk_zone  
WHERE event_dt >= DATEADD(day, -1, CURRENT_DATE())  
ORDER BY total_risk_score DESC  
LIMIT 10;
```

---

## Data quality, validation, and operations

### Data quality checks

#### Silver – silver\_vave\_eventsfeed.py

- Uses pyspark.pipelines expectations:
  - expect\_all\_or\_drop:
    - latitude BETWEEN -90 AND 90
    - longitude BETWEEN -180 AND 180

- risk\_score IS NOT NULL
  - event\_id IS NOT NULL
  - h3\_ix IS NOT NULL
- expect:
  - event\_timestamp IS NOT NULL
- Rows failing these checks are dropped and can be reported via pipeline metrics.

## Gold – reconciliation and sanity checks

- Silver vs Gold (cell):
  - For each event\_dt, SUM(event\_count) in main.gold.daily\_risk\_cell should match the count of Silver events for that day.
- Metric sanity:
  - total\_risk\_score should not be negative.
  - avg\_risk\_score should be within the expected business range (e.g. 0–100).
- Zone overlap:
  - SUM(total\_risk\_score) over all zones will exceed the cell-level total due to overlap; this is documented and expected.

Together, these checks validate geo-spatial correctness and detect bad data (invalid coordinates, nulls, inconsistent metrics).

## Scheduling and backfill

### Scheduling

- Bronze: run Vave\_Bronze\_dbt.ipynb on a schedule or trigger (e.g. new files landing) to append into main.bronze.vave\_eventsfeed.
- Silver: run silver\_vave\_eventsfeed.py as a structured streaming job over Bronze; CDC ensures only new/updated events are applied.
- Gold: run gold\_daily\_risk\_cell.py and gold\_daily\_risk\_zone.py on a schedule or as part of a pipeline triggered by Silver updates.

### Backfills

- Use widgets in Vave\_Config\_Params.ipynb (e.g. fil\_pth\_str, bch\_id) to:
  - Re-ingest specific historical files into Bronze.
  - Re-run Silver/Gold for selected date ranges (event\_dt) without disturbing other partitions.

## Scaling (10k vs 100M events/day)

- **10k events/day:**
  - Single H3 resolution; event\_dt partitioning is enough.
  - Modest cluster sizes; DQ checks run inline.
  - Simple scheduling and manual backfill are adequate.
- **100M events/day:**
  - Heavier partitioning and clustering (e.g. by event\_dt and potentially h3\_ix).
  - Multiple H3 resolutions to control cell counts.
  - Structured streaming with robust checkpointing for Bronze → Silver.
  - Automated dashboards/alerts for data quality and freshness (e.g. ingestion lag, null rates, distribution shifts).

---

## Known limitations and improvements

- Single H3 resolution and fixed zone definition ( $k=1$ ) – does not yet support multi-resolution or configurable neighbourhood size.
- Data-quality checks exist but are not yet surfaced in a dedicated monitoring dashboard.
- With more time, natural next steps are:
  - Multi-resolution H3 views (e.g. coarse and fine layers).
  - Richer DQ monitoring and alerting.
  - More advanced spatial analytics (e.g. neighbour similarity, radius queries) built on the same H3-indexed Silver layer.